Build your own Tokenizer

A 16 stage byte level BPE challenge

Build a small tokenizer that learns merge rules from a corpus, saves them, and uses them to turn new text into token IDs and back. The recommendation is 16 cumulative stages across roughly 6–10 days of flexible, AI-assisted work. Each stage adds one observable behavior, with concrete inputs, outputs, and tests. The time range is a planning estimate, not a measured completion forecast.

What learners will build

The finished system is a command-line byte-pair encoding tokenizer. It can train on a small corpus; encode unseen text using that learned model; decode IDs back to the original bytes; and save or load the model between processes. All 256 byte values have base tokens, so unfamiliar words and Unicode text still have a representation.

Its real-world analogue is the train–encode–decode core of a byte-level BPE tokenizer, such as the educational implementation in OpenAI's tiktoken project. Our course deliberately specifies its own small model format and merge rules. Its IDs are meaningful only with its own model; they cannot be sent to an existing LLM as that model's tokenizer output. [5]

Learners finish with a working algorithm they can inspect and explain. The core requires no neural network, GPU, paid API, or large dataset. Regular language libraries for JSON, files, and UTF-8 are fine; calling a tokenizer library to perform the core algorithm would skip the learning goal.

The finished workflow

Train three merges:

```
./your_program.sh train --text 'banana banana' --merges 3
{"version":1,"base_vocab_size":256,"merges":[[97,110],[98,256],[256,97]]}
```

Save the same model:

```
./your_program.sh train --text 'banana banana' --merges 3 --output model.json
```

The same model JSON is printed and written to model.json.

Encode new text:

```
./your_program.sh encode --model model.json --text 'banana!'
[257,258,33]
```

Decode those IDs:

```
./your_program.sh decode --model model.json --ids '[257,258,33]'
banana!
```

In this example, token 256 is an, 257 is ban, and 258 is ana. The 13-byte training text becomes five token IDs after three merges. These IDs describe tokens; JSON file size is a separate concern.

How this matches the existing challenges

The public Shell definition separates displaying a prompt, rejecting an unknown command, running a REPL, and adding individual built-ins. Its directory-changing work is split by absolute paths, relative paths, and the home directory. Redis separately tests one fixed PING response, repeated commands on one connection, and concurrent clients. These are narrow behavioral increments, not day-sized modules. [1–4]

The repositories put ordered stage metadata in course-definition.yml and learner-facing instructions in stage_descriptions. Metadata includes names, slugs, and difficulty; extensions are represented separately. Instruction length varies: Shell's first stage is brief, while Redis stages often include an explicit Tests section and scoped notes. There is no single mandatory long stage template. [1–4]

The proposal below adopts that granularity: one growing executable, a small new behavior per stage, a single main concept, exact examples, and objective checks. Pair counting and pair replacement are exposed through two teaching commands so learners can inspect the algorithm before combining it. These commands stay available as diagnostics. The finished application's everyday workflow is train, encode, and decode.

Shared behavior across the course

Byte IDs: IDs 0–255 are the corresponding byte values. Convert --text to UTF-8 bytes without normalization, trimming, case changes, or added separators. The whole input is one sequence, including spaces and newlines; there is no word splitting.

Training: Count every adjacent window, including overlapping windows. Choose the pair with highest count; break ties by the numerically smallest (left ID, right ID). Assign output ID 256 + merge index, starting at index 0. Replace all occurrences left-to-right without overlap, recount, and repeat.

Stopping: --merges N is a maximum number of new merges, not a target vocabulary size. Stop after N merges or when no pair has count at least 2. Empty input and N = 0 produce an empty merge list. This minimum-frequency rule is a course design choice.

Encoding: Start with base byte IDs. Among learned pairs present in the current sequence, choose the one with the lowest saved merge index, apply it left-to-right without overlap, and repeat until none applies. Input frequency does not choose encoding priority. A learned pair may apply even when it occurs only once in this new input.

Decoding: Expand every ID into its byte sequence, concatenate all bytes, and write those exact bytes to stdout. Do not decode each token separately: a token may contain only part of a UTF-8 character. Valid UTF-8 input therefore round-trips exactly.

Command output: encode, pairs, merge, and train print one JSON value plus a newline. JSON whitespace and object-key order do not matter; array order does. decode emits raw bytes with no added newline. Debug logging belongs on stderr. Examples show terminal output without representing its framing newline.

Training input: train accepts --text TEXT from its first training stage. Stage 10 adds --input PATH as an alternative, reading the file's bytes unchanged. Exactly one input source is required. Example strings and corpus files have no trailing newline unless stated.

The 16 cumulative stages

Complete these in order. Every stage preserves earlier behavior. Inputs are valid within the current stage's scope until the explicit error-handling stage. Examples use the same launcher as the public CodeCrafters stage instructions. Each stage names its main concept; tests may also exercise earlier concepts.

1 Encode ASCII text

New capability: Add encode --text for ASCII input. With no model, emit one integer ID per byte in order. This is the first runnable slice of the final encoder.

Concept: A token ID can name a byte.

Input: `./your_program.sh encode --text 'Hi!'`

Output: `[72,105,33]`

Objective tests:

- Check 'A', 'hello', spaces, repeated characters, and punctuation against exact byte arrays.
- Empty text produces []; preserve order and every whitespace byte. Non-ASCII input is introduced next.

2 Encode UTF8 bytes

New capability: Extend the same command to all valid UTF-8 text. Keep IDs in 0–255, including languages and emoji that use several bytes per character.

Concept: Characters and bytes are different units.

Input: `./your_program.sh encode --text 'café'`

Output: `[99,97,102,195,169]`

Objective tests:

- '🙂' produces [240,159,153,130]. Mixed ASCII and Unicode preserve exact byte order.
- Precomposed 'é' and decomposed 'e' plus U+0301 produce [195,169] and [101,204,129] respectively. Do not normalize them.

3 Decode base token IDs

New capability: Add decode --ids for a JSON array of base IDs. Concatenate the corresponding bytes and write them directly to stdout, with no extra newline.

Concept: Lossless decoding reconstructs bytes.

Input: `./your_program.sh decode --ids '[99,97,102,195,169]'`

Output: `café`

Objective tests:

- Decode [] to zero bytes and [0,10,255] to exact bytes 00 0A FF, using byte comparison rather than terminal rendering.
- Encode then decode valid UTF-8 fixtures, including emoji and embedded newlines; compare the original bytes exactly. Unknown-ID errors are added in stage 16.

4 Count adjacent pairs

New capability: Add the diagnostic command pairs --ids. Return [left,right,count] records sorted by numeric (left,right), regardless of count. Count overlapping windows too.

Concept: Pair frequency is counted over adjacent windows.

Input: `./your_program.sh pairs --ids '[97,98,97,98]'`

Output: `[[97,98,2],[98,97,1]]`

Objective tests:

- [97,97,97] produces [[97,97,2]], although only one replacement will later fit.
- [] and [97] produce []; [256,97,256,97] produces [[97,256,1],[256,97,2]]. Pair counting operates on IDs, not characters.

5 Replace one pair

New capability: Add merge --ids --pair --new-id. Scan left-to-right; when the chosen pair matches, append the new ID and advance by two. Otherwise copy one ID. This command does not train or choose a pair.

Concept: Replacement consumes a non-overlapping pair.

Input: `./your_program.sh merge --ids '[97,97,97]' --pair '[97,97]' --new-id 256`

Output: `[256,97]`

Objective tests:

- Four 97s become [256,256]; five become [256,256,97]. Never reuse a consumed element.
- An absent pair leaves input unchanged. Replace every occurrence in [97,98,99,97,98], obtaining [256,99,256]. Keep empty input empty.

6 Train the first merge

New capability: Add train --text --merges 1. Count pairs, choose the uniquely most frequent pair, assign ID 256, and replace it using the previous stage. Emit a versioned model object. This stage uses a unique winner with count at least 2.

Concept: Training learns a reusable rule from frequency.

Input: `./your_program.sh train --text 'abab' --merges 1`

Output: `{"version":1,"base_vocab_size":256,"merges":[[97,98]]}`

Objective tests:

- 'abab' learns [97,98]; 'zzzzz' learns [122,122], whose replacement sequence is [256,256,122].
- Use randomized two-byte patterns with a unique winner; inspect the resulting model. The replacement primitive remains independently testable through merge.

7 Break training ties

New capability: Extend the selection rule to tied frequencies. Compare token IDs numerically: smaller left ID wins, then smaller right ID. Do not rely on map iteration order or whichever pair appeared first.

Concept: Deterministic tie rules make models reproducible.

Input: `./your_program.sh train --text 'babab' --merges 1`

Output: `{"version":1,"base_vocab_size":256,"merges":[[97,98]]}`

Objective tests:

- Both [98,97] and [97,98] occur twice; [97,98] must win even though [98,97] appears first.
- 'acababac' gives [97,98] and [97,99] count 2 each; choose [97,98]. Repeat training in fresh processes and require identical merge arrays.

8 Learn successive merges

New capability: Allow --merges N for examples with enough repeated pairs. After each replacement, recount the current ID sequence and append a new rule. New rules can reference previously learned IDs.

Concept: Each merge changes the next frequency table.

Input: `./your_program.sh train --text 'banana banana' --merges 3`

Output: `{"version":1,"base_vocab_size":256,"merges":[[97,110],[98,256],[256,97]]}`

Objective tests:

- After each step the sequences are [98,256,256,97,32,98,256,256,97], then [257,256,97,32,257,256,97], then [257,258,32,257,258].
- Compare 1-, 2-, and 3-merge runs. Reusing the original frequency table must fail. Verify learned IDs are exactly 256, 257, 258.

9 Stop training safely

New capability: Stop when the merge budget is exhausted or the best count is below 2. Support zero merges, empty input, and one-byte input without inventing a rule.

Concept: A requested budget is an upper bound.

Input: `./your_program.sh train --text 'abc' --merges 10`

Output: `{"version":1,"base_vocab_size":256,"merges":[]}`

Objective tests:

- Empty text and --merges 0 produce empty merge arrays. 'aaaaa' with budget 10 learns only [[97,97]].
- 'banana banana' with budget 99 learns four rules: [[97,110],[98,256],[256,97],[257,258]]. Stop at [259,32,259], whose remaining pair counts are 1.

10 Save a trained model

New capability: Add train --input PATH as a file alternative to --text, and --output PATH to persist the model. Read corpus bytes unchanged. Keep printing the same model JSON so existing behavior remains usable.

Concept: A learned model is data that outlives a process.

Input: `./your_program.sh train --input corpus.txt --merges 3 --output model.json`

Output: With corpus.txt containing exactly banana banana: stdout and model.json contain the three-merge JSON from stage 8.

Objective tests:

- Parse both outputs and require equal version, base vocabulary, and ordered merges. Confirm the file exists after the trainer exits.
- The same bytes via --text and --input learn the same model. A corpus containing a newline preserves that byte; do not silently strip it. No tokens or frequency tables need to be stored.

11 Load the model contract

New capability: Add --model PATH to encode. Read version, base_vocab_size, and the ordered merge list. First support an empty model; encoding remains byte-for-byte identical to the no-model path. Nonempty model behavior follows next.

Concept: A serialized model defines a stable boundary.

Input: `./your_program.sh encode --model empty.json --text 'Hi!'`

Output: `[72,105,33]`

Objective tests:

- empty.json is {"version":1,"base_vocab_size":256,"merges":[]}. Load it in a fresh process with no training state.
- Accept ordinary JSON whitespace and reordered object keys; the empty model behaves like no model. A missing/unreadable model or malformed model JSON must exit nonzero, proving --model is read. Exact error codes arrive in stage 16.

12 Apply a learned pair

New capability: Extend --model encoding to one learned pair. Start with byte IDs and apply its non-overlapping replacement, even if that pair appears only once in the new text. Preserve bytes not covered by the rule.

Concept: Inference reuses a rule on unseen input.

Input: `./your_program.sh encode --model ab.json --text 'zab!ab'`

Output: `[122,256,33,256]`

Objective tests:

- ab.json uses merges [[97,98]]. 'ab' produces [256]; 'z!' produces [122,33]; empty input stays [].
- A model with [[97,97]] encodes 'aaa' as [256,97]. Encoding must not create new rules or change the model file.

13 Respect learned merge priority

New capability: Choose among multiple applicable rules by their position in the saved merge list, never by their frequency in the new input. Apply the chosen pair and repeat. This stage uses rules whose operands are base IDs.

Concept: Encoding priority comes from saved rank.

Input: `./your_program.sh encode --model priority.json --text 'abc bc bc'`

Output: `[256,99,32,257,32,257]`

Objective tests:

- priority.json uses merges [[97,98],[98,99]]. The higher-priority 'ab' pair occurs once and 'bc' occurs three times initially; 'ab' must still merge first.
- Reverse the fixture’s rule order to [[98,99],[97,98]] and require [97,256,32,256,32,256]. Repeated encoding is deterministic, and no new frequency-based training occurs.

14 Encode hierarchical tokens

New capability: Allow learned pairs to contain earlier learned IDs. Re-evaluate the current adjacent sequence until no rule applies. This completes general encoding for the course model.

Concept: Newly formed tokens can enable further merges.

Input: `./your_program.sh encode --model model.json --text 'bana ana ana'`

Output: `[257,97,32,258,32,258]`

Objective tests:

- Use the three-merge banana model. After applying [97,110], pair [98,256] occurs once and [256,97] occurs three times; rank 1 still wins before rank 2.
- 'banana!' produces [257,258,33]. 'banana banana' produces [257,258,32,257,258]. Check inputs where newly formed tokens enable two or more subsequent levels.

15 Decode learned tokens

New capability: Add --model to decode. Build byte expansions in merge order by concatenating each rule's two earlier expansions. Concatenate the requested token expansions before writing raw bytes. This completes the learned-model round trip.

Concept: A token's meaning is a byte sequence.

Input: `./your_program.sh decode --model model.json --ids '[257,258,33]'`

Output: `banana!`

Objective tests:

- Base and learned IDs can mix. The banana model maps 256 to 'an', 257 to 'ban', and 258 to 'ana'.
- With merges [[169,195]], encoding 'éé' gives [195,256,169]; decoding recovers bytes C3 A9 C3 A9. Token 256 alone is not valid UTF-8. Also train on 'éé', which learns [[195,169]], and round-trip emoji and unseen multilingual text.

16 Reject invalid inputs cleanly

New capability: Add a consistent failure contract: exit status 1, empty stdout, and one JSON error object on stderr. Validate CLI input, token IDs, files, and model structure before emitting results. Valid operations continue to exit 0.

Concept: Validate a model before trusting its token graph.

Input: `./your_program.sh decode --model model.json --ids '[999]'`

Output: stdout: empty; stderr: {"error":"unknown_token"}; exit: 1

Objective tests:

- Unknown decode ID → unknown_token; malformed CLI JSON or non-integer/negative command-line IDs → invalid_input; unreadable model/corpus or unwritable output → io_error.
- Wrong version/base size, malformed model JSON or merge entries, invalid IDs inside model rules, duplicate pair rules, or a rule referencing its own or a future ID → invalid_model. Negative/non-integer merge budgets or conflicting/missing input flags → invalid_input. Require no success output on failure.

The model format and scope boundary

The model is a JSON object with version equal to 1, base_vocab_size equal to 256, and merges as an ordered array of two-integer arrays. Entry i means the pair becomes token 256 + i, and i is its rank. Its operands must be nonnegative IDs smaller than 256 + i. Each pair may appear only once. Unknown extra object fields may be ignored; required fields and types must be checked.

The order is essential: the banana model's [[97,110],[98,256],[256,97]] defines 'an', 'ban', and 'ana'. Stored frequencies, textual token spellings, and a separate vocabulary table are unnecessary. The base vocabulary and ordered rules are enough to reconstruct every token's bytes. Distinct IDs with identical byte expansions need not be rejected; identity follows the explicit pair rules.

This is an explicit pair-rule format. Loading arbitrary tiktoken token-byte vocabularies or reproducing GPT token IDs is outside the contract. No special tokens, regex pre-tokenization, normalization, chat templates, padding, truncation, streaming, or optimized trainer is included. Keep those as possible separate extensions after this core is validated.

A flexible 6 to 10 day path

Days 1–2: Stages 1–5: byte round trip and the two inspectable primitives.

Days 3–4: Stages 6–9: one training step, deterministic ties, repeated learning, and termination.

Day 5: Stages 10–12: save, load, and reuse a model in a fresh process.

Days 6–7: Stages 13–15: rank-based encoding, hierarchical merges, and learned-token decoding.

Days 8–10 as needed: Stage 16, regression tests, debugging, and explaining the full algorithm. Faster learners can combine the earlier sessions; difficult stages can take a full session.

How the course is tested

Stage tests should invoke ./your_program.sh as a black-box process, parse JSON structurally, compare decode output as bytes, and check exit status. Keep all earlier passing tests active. Fixtures should vary their bytes and text so hardcoded examples do not pass. Each test failure should identify the first incorrect intermediate behavior, not only report a final mismatch.

At the end, train and save a model, terminate the process, then encode and decode held-out text in new processes. Require exact byte equality for empty text, ASCII, newlines, accented text, emoji, and unseen scripts. Repeat training to confirm identical merge order. A supplied valid-model fixture must work too; encode cannot depend on trainer memory.

Use tiny deterministic examples to teach correctness, then a modest text corpus, for example 1–16 KiB with 16–128 requested merges, to demonstrate useful reuse. Do not require a compression percentage on arbitrary text or a production throughput target. Fewer token IDs is not proof of a smaller serialized file.

How to use AI while learning

AI can help write scaffolding, diagnose failures, and propose tests. Before running generated code, the learner predicts the next pair, merge result, or encoded IDs for a tiny fixture. After a stage passes, they explain why it passes and add one adversarial case. The goal is understanding an algorithm while building it.

Three completion questions anchor the course: Why can three identical bytes contain two pair windows but only one replacement? Why can a rare pair merge before a common pair when encoding? Why must bytes be joined before UTF-8 decoding? A learner who can trace those cases has learned the mechanisms the challenge is designed to teach.

Sources

Public main-branch sources consulted on 19 October 2026. Repository contents can change. The stage design, model format, and time estimate above are proposed course decisions, not claims that CodeCrafters already teaches this tokenizer.

[1] CodeCrafters Shell course definition

[2] CodeCrafters Shell first stage

[3] CodeCrafters Redis course definition

[4] CodeCrafters Redis repeated PING stage

[5] OpenAI tiktoken educational BPE implementation